API Models Guide

Chat, vision, tool calling, embeddings and speech-to-text through one OpenAI-compatible API

BASE URL

```
https://spod.ai-factory.fastweb.it/v1
```

Contents

1. Introduction 3
2. Authentication 3
3. Models 4
4. Chat completions with tool calling 5
5. Responses API with tool calling 7
6. Vision (image input) 8
7. Streaming 9
8. Embeddings 10
9. Speech-to-text (transcription) 10
10. Errors 11

Introduction

The API is OpenAI-compatible. Any OpenAI SDK or plain HTTP client works: point it at the base URL and send your API key.

Base URL: `https://spod.ai-factory.fastweb.it/v1`

Endpoint	Method	Use
`/v1/chat/completions`	POST	Chat, vision, tool calling, streaming
`/v1/responses`	POST	Same features with the Responses API format
`/v1/embeddings`	POST	Text and image embeddings
`/v1/audio/transcriptions`	POST	Speech-to-text
`/v1/models`	GET	List the models available to your key

Authentication

Send your API key in the `Authorization` header of every request. The examples read it from an environment variable:

```
export API_KEY="your-api-key"
```

```
Authorization: Bearer $API_KEY
```

Keep your key secret. If a key leaks, revoke it in the dashboard and create a new one. All the keys of an account share the same credit balance.

Models

The model id is the value of the `model` field in each request.

Model	Model id	Type	Hosting	Availability
Qwen 3.6	`Qwen/Qwen3.6-35B-A3B-FP8`	Chat, vision, tool calling	International (outside EU)	Listed until December 2026
GLM-5.3-flash	`glm-5.3-flash`	Chat, vision, tool calling	Outside EU (China/US); not available with EU hosting	From 12 October 2026
Gemma 4 26B-A4B	`google/gemma-4-26B-A4B-it`	Chat, vision, tool calling	EU	From 12 October 2026
Qwen3.8 27B	`Qwen/Qwen3.8-27B`	Chat, vision, tool calling	Outside EU	From 12 October 2026
Qwen3-VL-Embedding-2B	`Qwen/Qwen3-VL-Embedding-2B`	Embeddings (text and image)	EU	Available
Whisper large v3 turbo	`openai/whisper-large-v3-turbo`	Speech-to-text	EU	Available

New models. GLM-5.3-flash, Gemma 4 26B-A4B and Qwen3.8 27B open on 12 October 2026.

End of listing. Qwen 3.6 is listed until December 2026. Plan to move to another chat model before then.

Capabilities

Model id	Text chat	Image input	Tool calling	Embeddings	Speech-to-text
`Qwen/Qwen3.6-35B-A3B-FP8`	✓	✓	✓	–	–
`glm-5.3-flash`	✓	✓	✓	–	–
`google/gemma-4-26B-A4B-it`	✓	✓	✓	–	–
`Qwen/Qwen3.8-27B`	✓	✓	✓	–	–
`Qwen/Qwen3-VL-Embedding-2B`	–	✓	–	✓	–
`openai/whisper-large-v3-turbo`	–	–	–	–	✓

The chat models work on both `/v1/chat/completions` and `/v1/responses`. Any chat model id can replace the one used in the examples below.

Chat completions with tool calling

The example is an order-tracking assistant. Your application exposes a `get_order_status` function; the model decides when to call it and with which arguments.

1. Send the request with the tool

```
curl https://spod.ai-factory.fastweb.it/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $API_KEY" \
  -d '{
    "model": "Qwen/Qwen3.6-35B-A3B-FP8",
    "messages": [
      {"role": "user", "content": "Where is my order A-10293?"}
    ],
    "tools": [{
      "type": "function",
      "function": {
        "name": "get_order_status",
        "description": "Returns the shipping status of an order",
        "parameters": {
          "type": "object",
          "properties": {"order_id": {"type": "string"}},
          "required": ["order_id"]
        }
      }
    }],
    "tool_choice": "auto"
  }'
```

2. Read the tool call

The response has `finish_reason: "tool_calls"`. The relevant part of `choices[0]`:

```
{
  "message": {
    "role": "assistant",
    "tool_calls": [{
      "id": "call_1",
      "type": "function",
      "function": {"name": "get_order_status", "arguments": "{\"order_id\": \"A-10293\"}"}
    }]
  },
  "finish_reason": "tool_calls"
}
```

`arguments` is a JSON string: parse it and run your function.

3. Send the result back

Repeat the conversation, add the assistant message with `tool_calls`, then a `tool` message whose `tool_call_id` matches the call. Send the same `tools` array as in step 1 (omitted here).

```
curl https://spod.ai-factory.fastweb.it/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $API_KEY" \
  -d '{
    "model": "Qwen/Qwen3.6-35B-A3B-FP8",
    "messages": [
      {"role": "user", "content": "Where is my order A-10293?"},
      {"role": "assistant", "tool_calls": [{
        "id": "call_1",
        "type": "function",
        "function": {"name": "get_order_status", "arguments": "{\"order_id\": \"A-10293\"}"}
      }]},
      {"role": "tool", "tool_call_id": "call_1",
       "content": "{\"status\": \"shipped\", \"delivery\": \"2026-10-16\"}"}
    ]
  }'
```

The model answers in plain text, for example "Your order A-10293 has shipped and should arrive on 16 October 2026."

Parallel tool calls

The model can request several calls in a single response. With the same request as step 1 and the question "Where are my orders A-10293 and B-20417?", `tool_calls` holds one entry per order:

```
{
  "tool_calls": [
    {"id": "call_1", "type": "function",
     "function": {"name": "get_order_status", "arguments": "{\"order_id\": \"A-10293\"}"}},
    {"id": "call_2", "type": "function",
     "function": {"name": "get_order_status", "arguments": "{\"order_id\": \"B-20417\"}"}}
  ]
}
```

Run each call and send back one `tool` message per call, each with its own `tool_call_id`.

Responses API with tool calling

The Responses API uses `input` instead of `messages`, and tools are flat: `name`, `description` and `parameters` sit directly on the tool object. The example is a scheduling assistant with a `create_calendar_event` function.

1. Send the request with the tool

```
curl https://spod.ai-factory.fastweb.it/v1/responses \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $API_KEY" \
  -d '{
    "model": "Qwen/Qwen3.6-35B-A3B-FP8",
    "input": "Book a project sync on 3 November 2026 at 10:00 CET.",
    "tools": [{
      "type": "function",
      "name": "create_calendar_event",
      "description": "Creates an event in the user calendar",
      "parameters": {
        "type": "object",
        "properties": {
          "title": {"type": "string"},
          "start": {"type": "string", "description": "ISO 8601 date-time"}
        },
        "required": ["title", "start"]
      }
    }],
    "store": false
  }'
```

2. Read the function call

The `output` array contains a `function_call` item:

```
{
  "type": "function_call",
  "call_id": "call_1",
  "name": "create_calendar_event",
  "arguments": "{\"title\": \"Project sync\", \"start\": \"2026-11-03T10:00:00+01:00\"}"
}
```

3. Send the result back

Responses are not stored (`store` is `false`), so send the earlier items again: the user message, the `function_call` item as returned, and a `function_call_output` item with the same `call_id` . Send the same `tools` array as in step 1 (omitted here).

```
curl https://spod.ai-factory.fastweb.it/v1/responses \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $API_KEY" \
  -d '{
    "model": "Qwen/Qwen3.6-35B-A3B-FP8",
    "input": [
      {"role": "user", "content": "Book a project sync on 3 November 2026 at 10:00 CET."},
      {"type": "function_call", "call_id": "call_1", "name": "create_calendar_event",
       "arguments": "{\"title\": \"Project sync\", \"start\": \"2026-11-03T10:00:00+01:00\"}"},
      {"type": "function_call_output", "call_id": "call_1",
       "output": "{\"event_id\": \"evt_7f3a21\", \"status\": \"created\"}"}
    ],
    "store": false
  }'
```

The model answers with a `message` output item, for example "Done: the project sync is booked for 3 November 2026 at 10:00 CET."

Vision (image input)

All chat models accept images. The examples pass a public HTTPS URL; a base64 data URI (`...`) works in the same field.

Chat completions

```
curl https://spod.ai-factory.fastweb.it/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $API_KEY" \
  -d '{
    "model": "google/gemma-4-26B-A4B-it",
    "messages": [{"role": "user", "content": [
      {"type": "text", "text": "Describe this image in one sentence."},
      {"type": "image_url", "image_url": {"url":
        "https://qianwen-res.oss-cn-beijing.aliyuncs.com/Qwen-VL/assets/demo.jpeg"}}
    ]}]
  }'
```

Responses API

The content parts are `input_text` and `input_image`, and `image_url` is a plain string.

```
curl https://spod.ai-factory.fastweb.it/v1/responses \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $API_KEY" \
  -d '{
    "model": "Qwen/Qwen3.8-27B",
    "input": [{"role": "user", "content": [
      {"type": "input_text", "text": "Describe this image in one sentence."},
      {"type": "input_image", "image_url":
        "https://qianwen-res.oss-cn-beijing.aliyuncs.com/Qwen-VL/assets/demo.jpeg"}
    ]}]
  }'
```

Streaming

Set `"stream": true` to receive the answer as server-sent events. `curl -N` turns off output buffering.

```
curl -N https://spod.ai-factory.fastweb.it/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $API_KEY" \
  -d '{
    "model": "glm-5.3-flash",
    "messages": [{"role": "user", "content": "Write a haiku about autumn."}],
    "stream": true
  }'
```

Each event is a `data:` line with a `chat.completion.chunk`, and the stream ends with `data: [DONE]`:

```
data: {"choices":[{"index":0,"delta":{"content":"Fog"}}]}
data: {"choices":[{"index":0,"delta":{"content":" on"}}]}
data: [DONE]
```

On `/v1/responses` the same flag produces typed events: text arrives in `response.output_text.delta` events and `response.completed` closes the stream.

Embeddings

`Qwen/Qwen3-VL-Embedding-2B` produces Matryoshka embeddings: `dimensions` sets the vector length, from 64 to 2048. Shorter vectors use less storage and search faster, at some loss of accuracy. Omit `dimensions` to get the full 2048-value vector.

```
curl https://spod.ai-factory.fastweb.it/v1/embeddings \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $API_KEY" \
  -d '{
    "model": "Qwen/Qwen3-VL-Embedding-2B",
    "input": ["Orders ship within two working days.", "Returns are accepted for 30 days."],
    "dimensions": 512
  }'
```

Response, with the vectors shortened:

```
{
  "data": [
    {"index": 0, "embedding": [0.0123, -0.0456, 0.0789]},
    {"index": 1, "embedding": [0.0234, -0.0111, 0.0567]}
  ]
}
```

The model also accepts image input, so text and images can be embedded in the same vector space.

Speech-to-text (transcription)

The request is `multipart/form-data`; curl sets the `Content-Type` header itself with `-F`. `language` (ISO 639-1 code such as `en` or `it`) is optional and improves accuracy.

```
curl https://spod.ai-factory.fastweb.it/v1/audio/transcriptions \
  -H "Authorization: Bearer $API_KEY" \
  -F file=@audio.mp3 \
  -F model=openai/whisper-large-v3-turbo \
  -F language=en
```

```
{"text": "Good morning, this is a test recording."}
```

`response_format` is optional: `json` (default), `text`, `verbose_json`, `srt` or `vtt`.

Errors

Errors follow the OpenAI format:

```
{"error": {"message": "Budget has been exceeded.", "type": "budget_exceeded", "code": "400"}}
```

HTTP status	Meaning	What to do
400 `budget_exceeded`	Prepaid credits exhausted	Top up your credits
400 (other types)	Invalid request: malformed JSON, unknown parameter, wrong model for the endpoint	Fix the request
401	Missing, invalid or revoked API key	Check the `Authorization` header and the key
403	Endpoint not available on the public API	Use one of the endpoints listed in this guide
429	Rate limit reached for the key	Retry with exponential backoff
503	Service temporarily unable to check credits or limits	Retry after a short wait